Name: Rahma Sabry Mohamed AbdRabo ID:9033

Name: Ahmed Tamer Abassy ID: 8842

1. Introduction

This lab implements a system that simulates TCP-like reliability over UDP, then builds an HTTP/1.0 server and client on top of it. UDP is a connectionless protocol that provides no guarantee of delivery, ordering, or error correction. The goal is to implement these guarantees entirely in the application layer.

The system is organized in three layers:

- Transport layer – ReliableUDP class implementing Stop-and-Wait ARQ, checksum verification, handshake (SYN/SYN-ACK/ACK), and connection teardown (FIN/ACK).
- Packet layer – JSON-based packet structure with sequence numbers, acknowledgment numbers, flags, data payload, and checksum.
- Application layer – HTTP/1.0 server and client supporting GET and POST methods, 200 OK and 404 NOT FOUND status codes.

2. Design & Implementation

2.1 Packet Structure

Each packet is serialized as a JSON object containing the following fields:

- seq – Sequence number of the sender.
- ack – Acknowledgment number (sequence being acknowledged).
- flags – List of control flags: SYN, SYN-ACK, ACK, DATA, FIN.
- data – The payload string (HTTP request or response).
- checksum – An 8-bit checksum computed over all other fields to detect corruption.

The checksum is calculated by encoding the concatenated fields as bytes and computing the sum modulo 256. Any packet with a mismatched checksum is silently dropped, causing the sender to timeout and retransmit.

2.2 Three-Way Handshake

Connection establishment follows the TCP three-way handshake pattern:

- Client sends SYN.
- Server responds with SYN-ACK.
- Client sends ACK. Connection is now established.

Each step includes timeout and retransmission logic. If a SYN-ACK is not received within the timeout period, the client resends the SYN automatically.

2.3 Stop-and-Wait Reliable Transfer

Data transfer uses the Stop-and-Wait (rdt 3.0) protocol. The sender transmits one DATA packet and waits for an ACK before sending the next. If the ACK is not received within the timeout window, the packet is retransmitted. The receiver drops duplicate packets (sequence number already acknowledged) but re-sends the ACK to unblock the sender.

2.4 Packet Loss & Corruption Simulation

The ReliableUDP class includes two simulation parameters that can be tuned at the top of reliable_udp.py:

- LOSS_RATE – Probability (0.0 to 1.0) that any DATA packet is silently dropped before sending.
- CORRUPT_RATE – Probability that a packet's checksum is deliberately incremented by 1, making it invalid at the receiver.

Both are set to 0.0 in normal operation. Setting them to non-zero values allows observation of retransmission behavior in Wireshark.

2.5 Connection Teardown

The client initiates teardown by sending a FIN packet. The server acknowledges with ACK and closes its socket. The client closes after receiving the ACK.

2.6 HTTP Layer

The HTTP layer is implemented in http_protocol.py and supports:

- GET – Requests a file by path. Server reads the file from disk and returns 200 OK with the content, or 404 NOT FOUND if the file does not exist.
- POST – Sends a body to the server. Server echoes it back with 200 OK.

The HTTP messages are passed as plain strings through the ReliableUDP send/receive interface, making the HTTP layer completely independent of the transport mechanism.

2.7 TCP Bridge for Postman Testing

Since Postman communicates over TCP and the server uses custom UDP, a tcp_bridge.py module was created. It listens on TCP port 7000, accepts a real HTTP connection from Postman, wraps the request through a full ReliableUDP handshake to the UDP server on port 8080, and returns the response over TCP. This enables testing with standard HTTP tools.

3. Simulation Results

3.1 Wireshark – UDP Traffic Capture

Wireshark was configured with the filter udp.port == 8080 on the loopback interface. The capture shows the complete packet exchange between client and server, including the SYN handshake, DATA packets carrying HTTP messages, ACK responses, and the FIN teardown sequence. The hex dump panel confirms the JSON packet format in the payload.

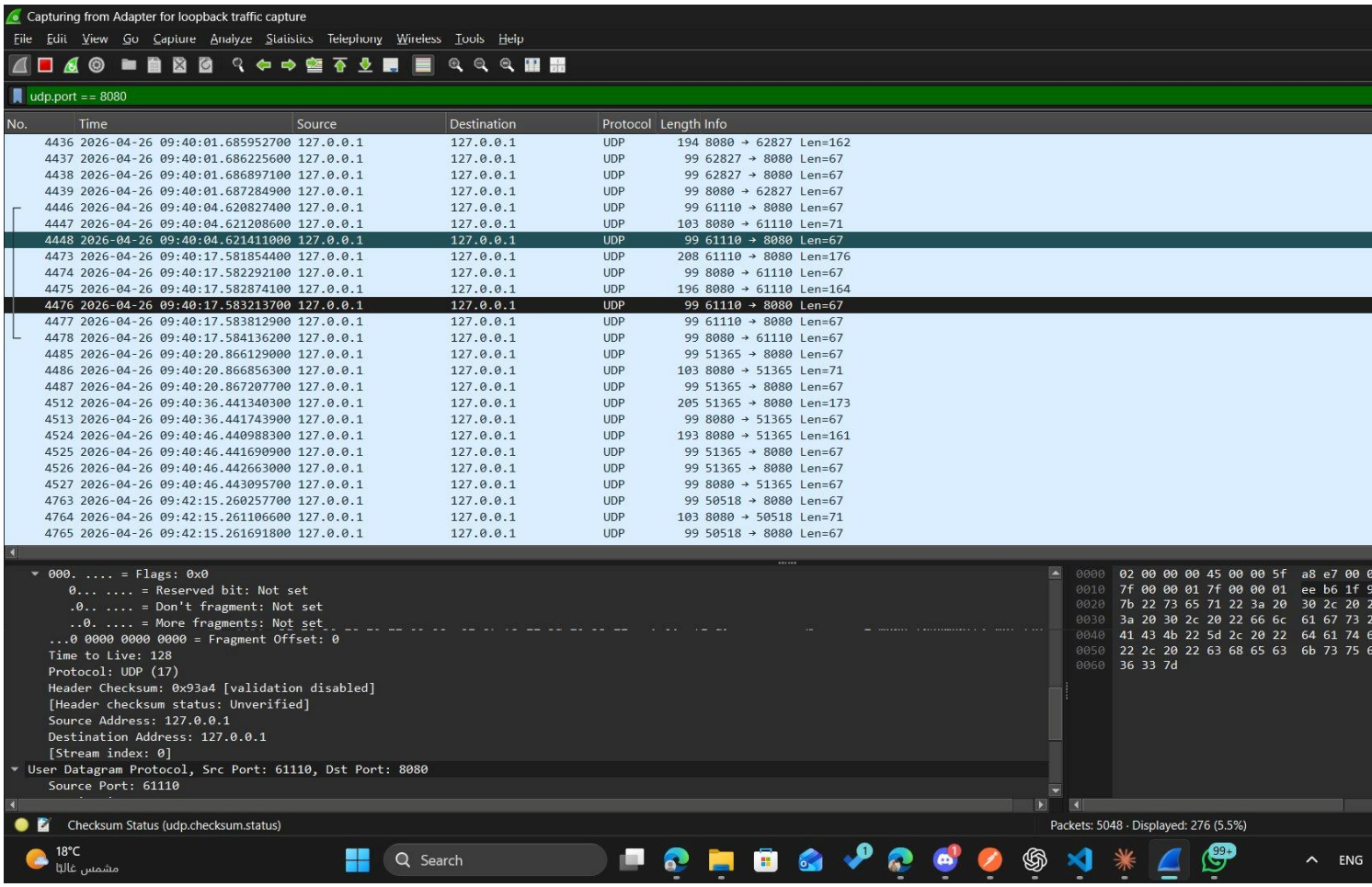

Figure 1 – Wireshark capture of ReliableUDP traffic on port 8080. Each UDP frame carries a JSON-encoded packet. The highlighted packet shows an ACK flag in the hex dump.

3.2 Postman – GET /index.html → 200 OK

A GET request was sent from Postman to http://127.0.0.1:7000/index.html via the TCP bridge. The server located the file on disk, read its contents, and returned HTTP/1.0 200 OK with the HTML body. Postman received and rendered the response correctly in 13 ms.

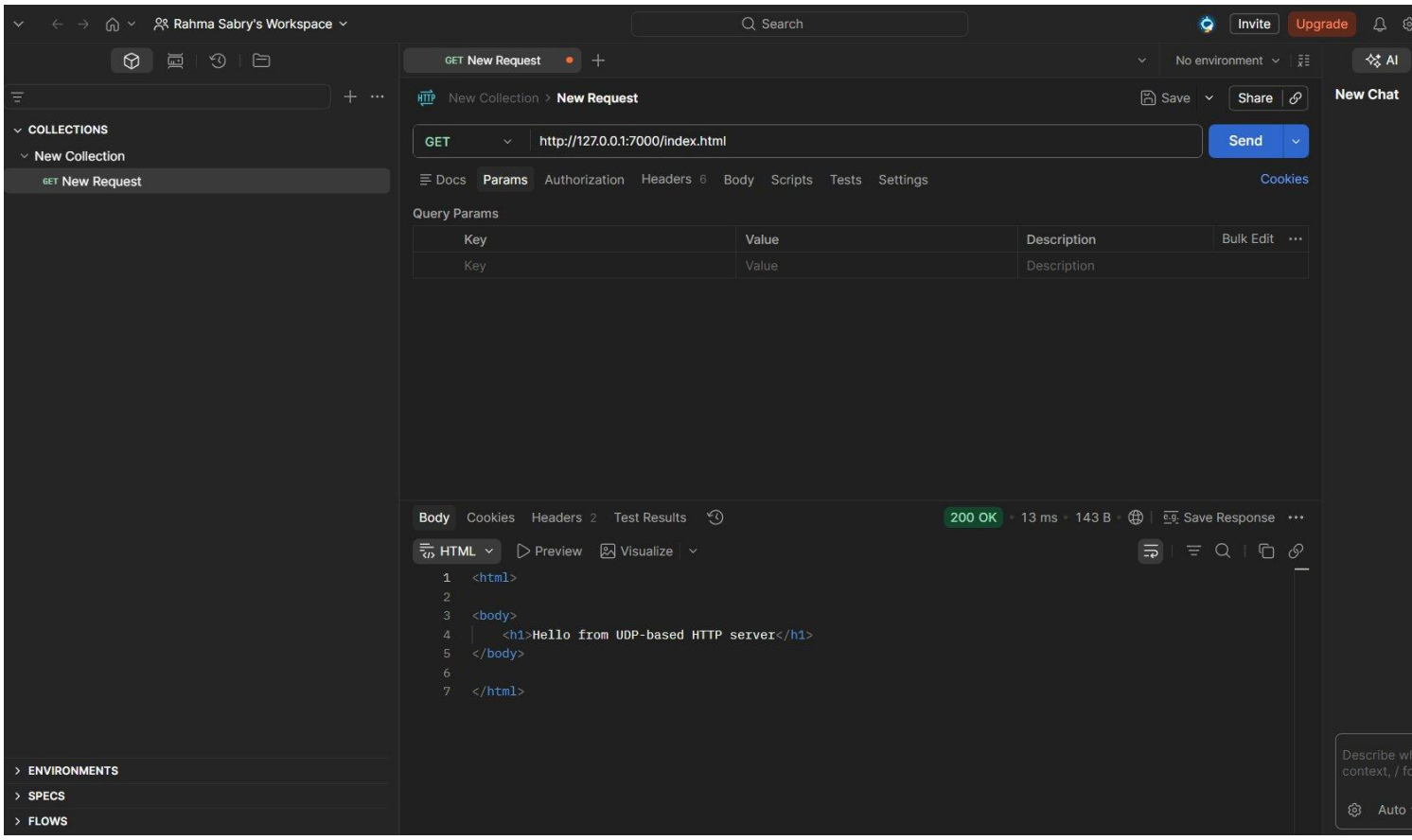

Figure 2 – Postman GET request to /index.html returns 200 OK with the HTML content of the file served by the UDP-based HTTP server.

3.3 Postman – GET /inex.html → 404 NOT FOUND

A GET request was sent to a non-existent path (/inex.html). The server searched for the file, did not find it, and returned HTTP/1.0 404 NOT FOUND with the body 'File not found'. This confirms correct error handling for missing resources.

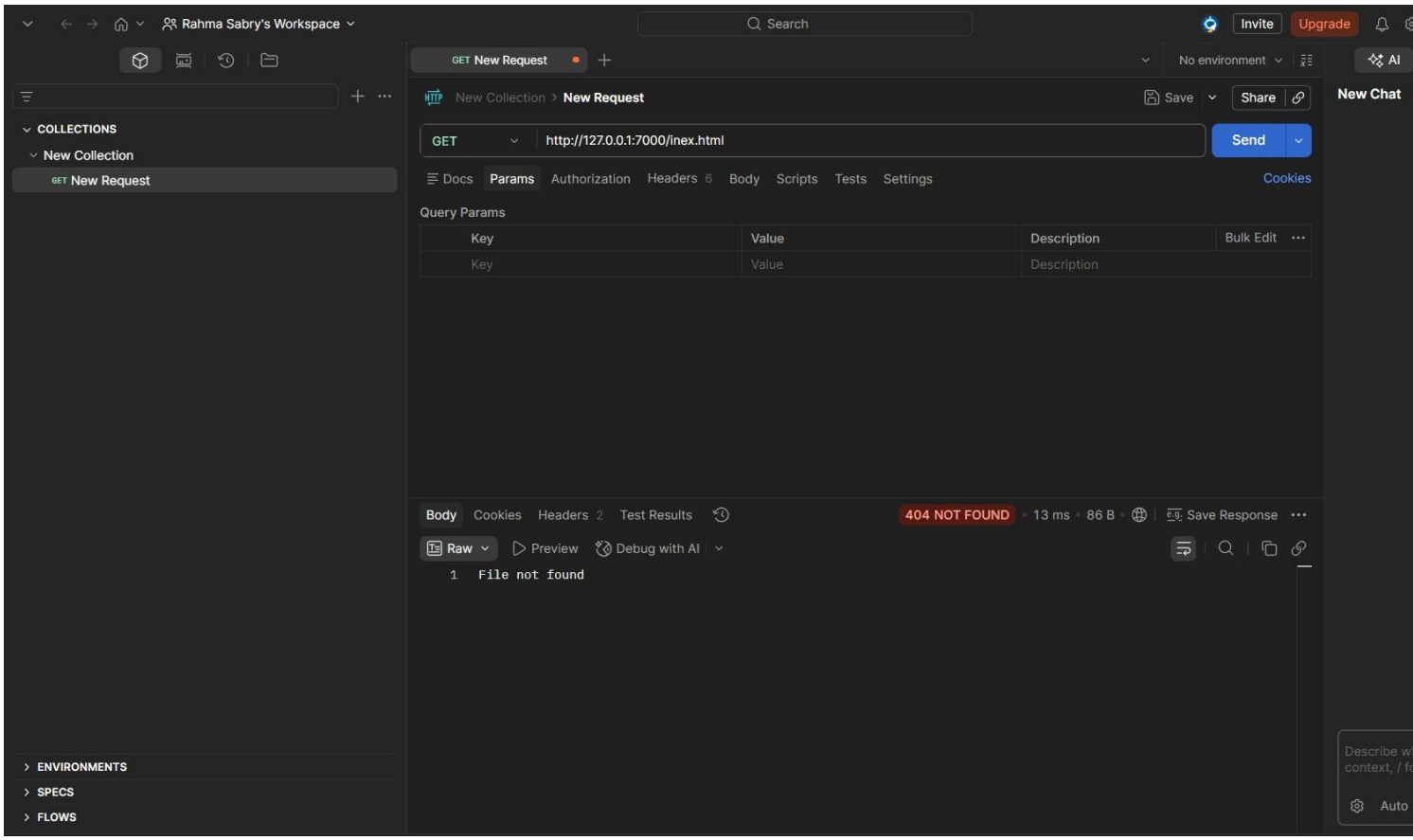

Figure 3 – Postman GET request to a non-existent path returns 404 NOT FOUND, confirming the server correctly handles missing files.

3.4 Postman – POST /index.html → 200 OK

A POST request was sent to http://127.0.0.1:7000/index.html. The server accepted the request and returned 200 OK with the echoed body. This confirms that the POST method is handled correctly regardless of the path.

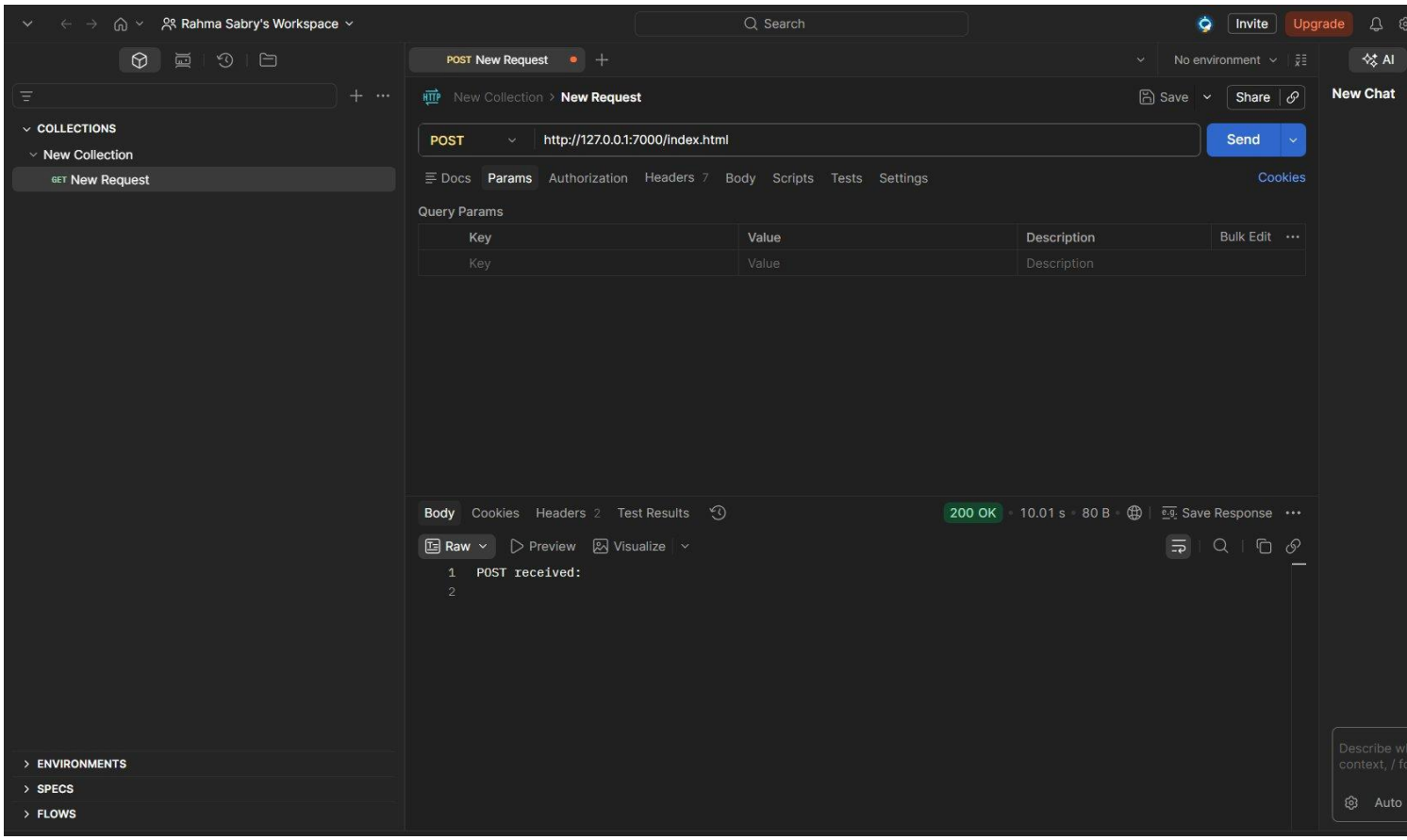

Figure 4 – Postman POST request returns 200 OK. The server echoes the received body, confirming POST method support.

4. Conclusion

This lab successfully demonstrated the implementation of a reliable transport protocol over UDP using Stop-and-Wait ARQ with checksum verification, sequence numbering, and TCP-like handshake and teardown. The HTTP/1.0 layer built on top of it correctly handles GET and POST requests with appropriate status codes.

The system was validated using three tools:

- The built-in Python test suite (test.py) confirmed the correctness of packet creation, checksum validation, and HTTP message parsing.
- Wireshark confirmed the packet-level exchange on the wire, showing SYN, DATA, ACK, and FIN packets in the correct order.
- Postman confirmed end-to-end HTTP behavior through the TCP bridge, returning 200 OK for valid GET and POST requests and 404 NOT FOUND for missing files.